Music Streaming Analysis Using Spark Structured APIs

1. Introduction

This project performs analytical processing on simulated music streaming data using Apache Spark Structured APIs (PySpark). The objective is to analyze user listening behavior and extract meaningful insights such as favorite genres, listening patterns, and loyalty metrics. The project demonstrates the use of Spark DataFrames, joins, aggregations, window functions, and time-based filtering.

2. Dataset Description

2.1 listening_logs.csv

This dataset contains user listening activity.

Columns:

• user_id – Unique identifier for each user

• song_id – Unique identifier for each song

• timestamp – Date and time of listening

• duration_sec – Duration of the song in seconds

2.2 songs_metadata.csv

This dataset contains song-related information.

Columns:

• song_id – Unique song identifier

• genre – Genre of the song

• artist – Artist name

• release_year – Year of release

The two datasets are joined using the song_id column.

3. System and Tools Used

• Python 3.x

• Apache Spark

• PySpark

• Spark Structured APIs

4. Project Structure

• main.py – Main Spark application

• input_generator.py – Script to generate input data

• songs_metadata.csv – Song dataset

• listening_logs.csv – User listening dataset

• requirements.txt – Python dependencies

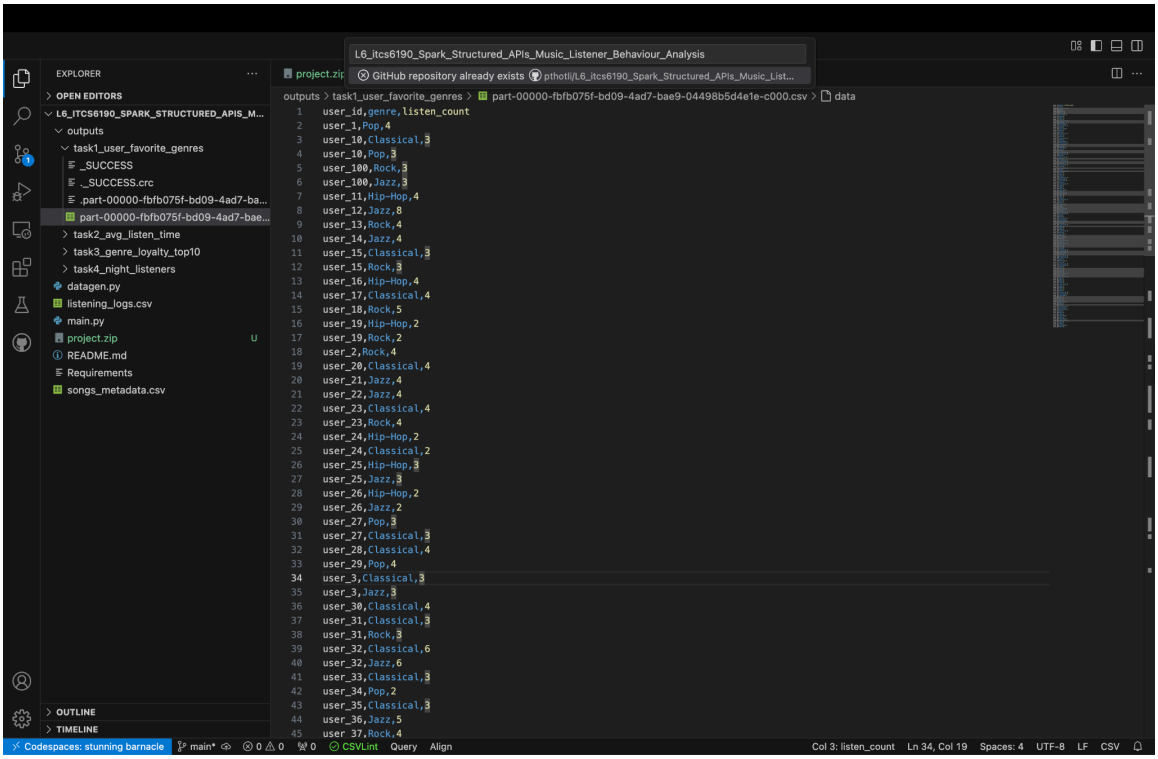
• outputs/ – Directory containing generated results

5. Tasks Implemented

5.1 User Favorite Genres

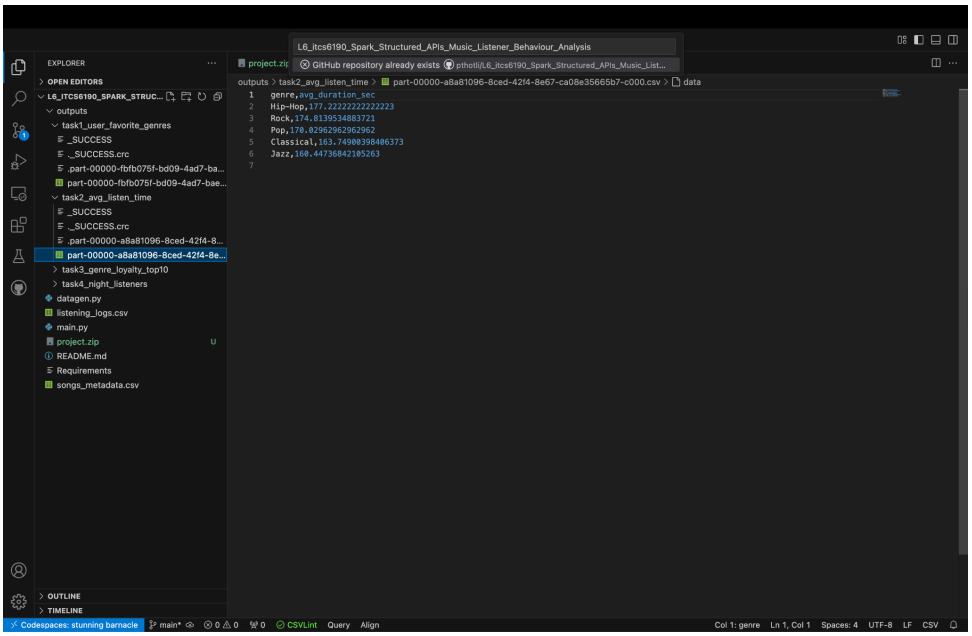
Identifies each user's most frequently listened genre using groupBy and window ranking functions.

Output:



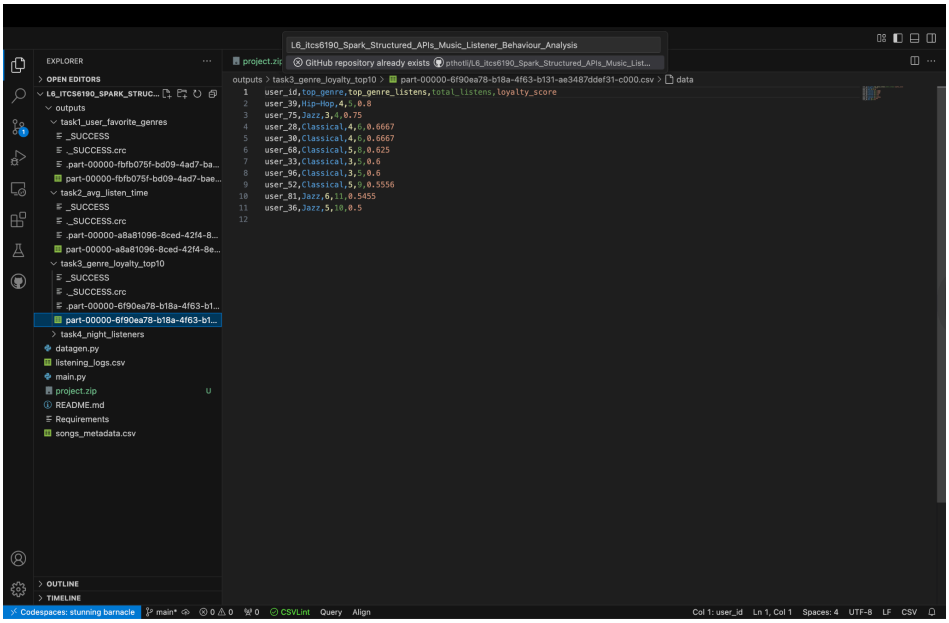
5.2 Average Listen Time per Genre

Calculates the average listening duration for each genre using aggregation functions.

Output: 

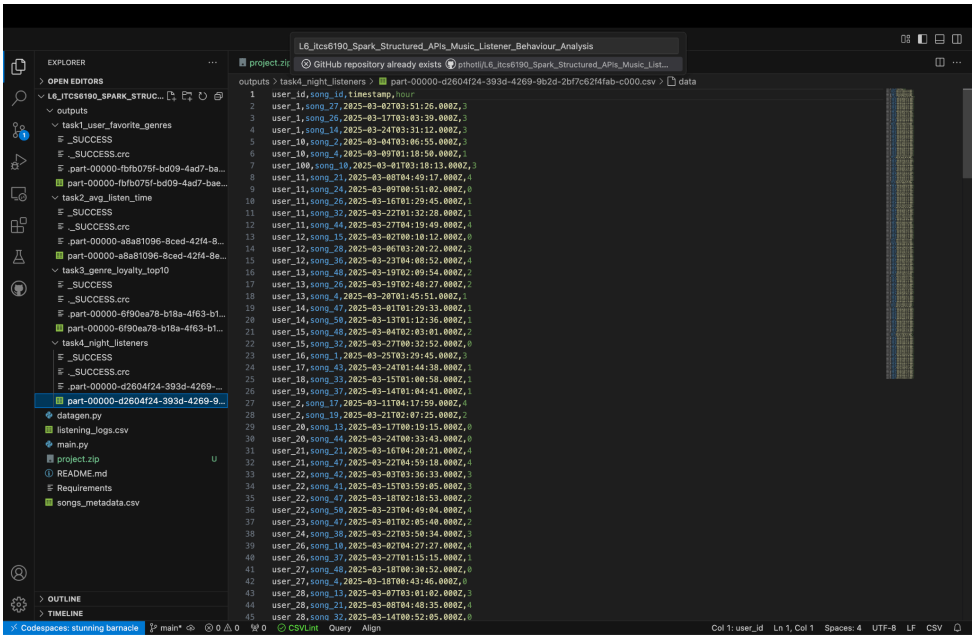
5.3 Top 10 Genre Loyalty Scores

Computes loyalty score as top_genre_listens divided by total_listens and identifies the top 10 users.

Output: 

5.4 Night Listening Behavior (12 AM – 5 AM)

Identifies users who listen to music between midnight and 5 AM using timestamp filtering.

Output: 

6. Execution Instructions

Step 1: Install prerequisites (Python, Spark, PySpark).

Step 2: Generate input data using python3 input_generator.py

Step 3: Run the Spark application using spark-submit main.py

Step 4: Verify outputs using: ls outputs/

7. Error Handling

• Ensured Spark is installed and added to PATH if spark-submit is not found.

• Install Java 8 or higher if Java errors occur.

• Delete existing outputs directory using rm -rf outputs.

8. Conclusion

This hands-on demonstrates how Apache Spark Structured APIs can efficiently process and analyze streaming data. By applying DataFrame transformations, joins, window functions, and aggregations, meaningful insights into user listening behavior are generated.